

Lecture 19: Cross-validation, Decision Trees

CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

- PA4, WA4, Q10 graded; PA4, Q10 solutions available
 - PA5, WA5 due next Sunday April 12
- 

A note on validation

- After we find the best hyperparameters by training multiple models on the training data (that doesn't include the validation set) and testing on the validation set, we create a final model by training on the “full training” set (training+validation). Then we test with testing data
 - We didn't specify this before.
 - The Week 10 notebook also overlooked this. This has now been fixed.
- 

Cross-validation

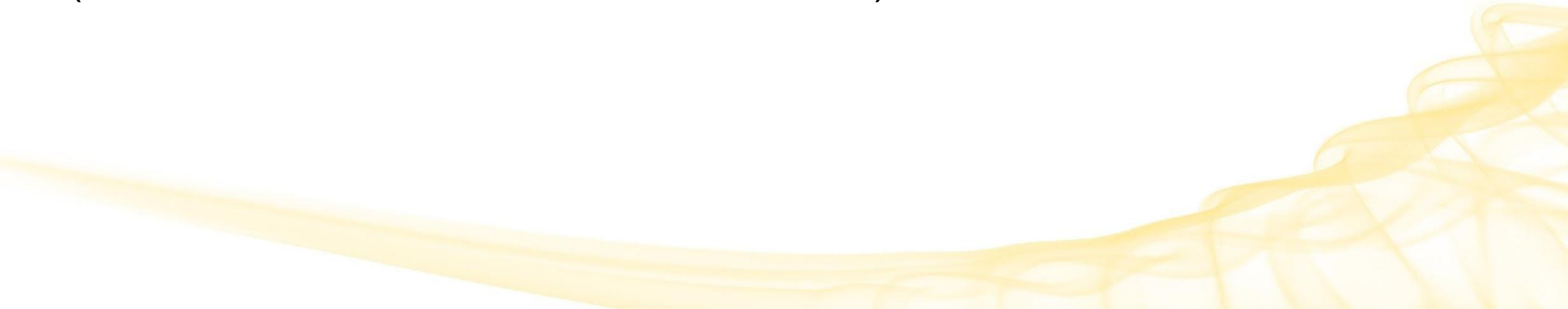
The main purposes of validation

are model selection (hyperparameter search)

and performance estimation (predicting how good the model will be on test data).

Cross-validation is a generalization of validation.

(Validation is k -fold cross-validation for $k=1$).

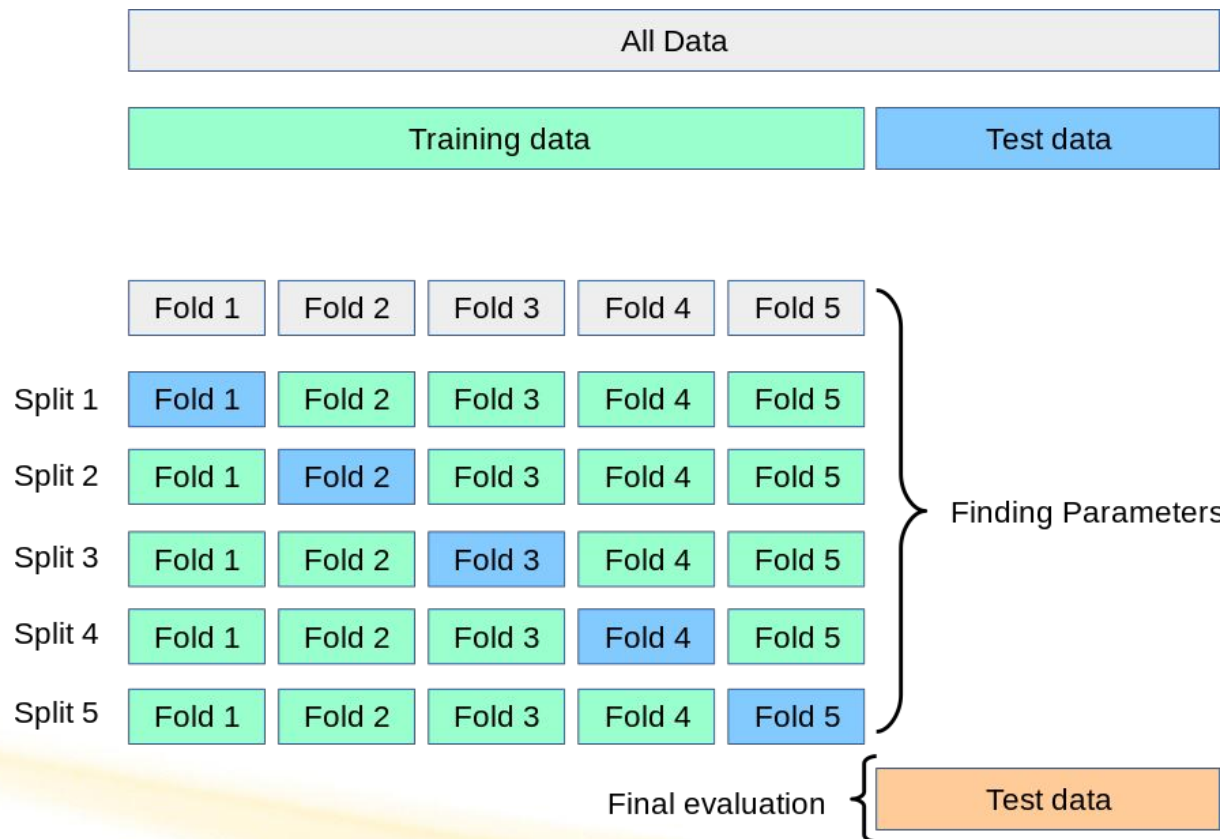


Types of Cross-Validation

- K-Fold Cross-Validation
 - Leave-One-Out Cross-Validation (LOOCV)
 - Stratified Cross-Validation
 - Many more...
- 

K-Fold Cross-Validation

Example for k=5:



Dataset Splitting: Data is divided into subsets, called **folds**.

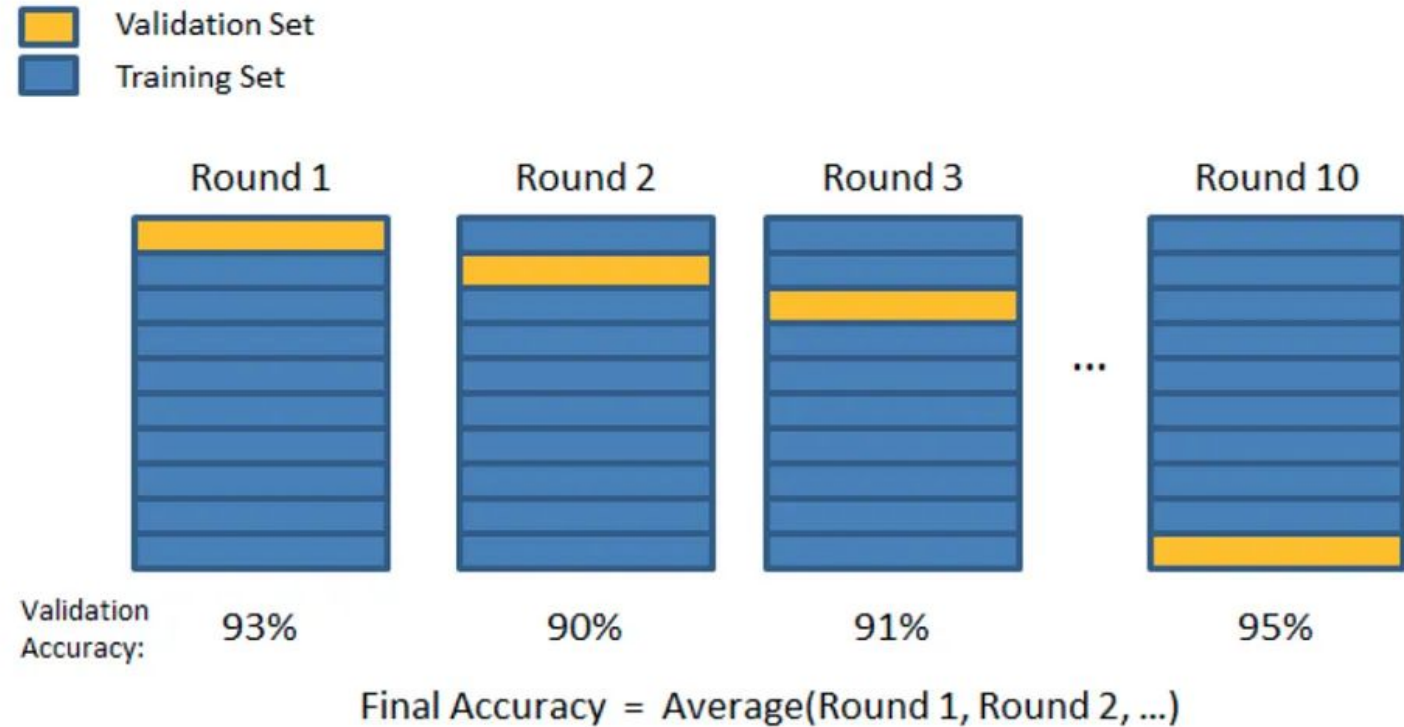
Training and Validation: Model is trained and validated on different folds. One fold used as validation set, others for training.

Repetition: Process repeats with each fold as the validation set.

Performance Assessment: Evaluates models across all folds. Ensures the model generalizes well to new data (avoid overfitting).

K-fold cross validation example for k=10

- Divide the dataset into k roughly equal-sized subsets (called **folds**)
- Train the model on k-1 of these folds, and test on the remaining one.
- Repeat k times, with each fold serving as the validation set exactly once.



How do we choose the number of folds K ?

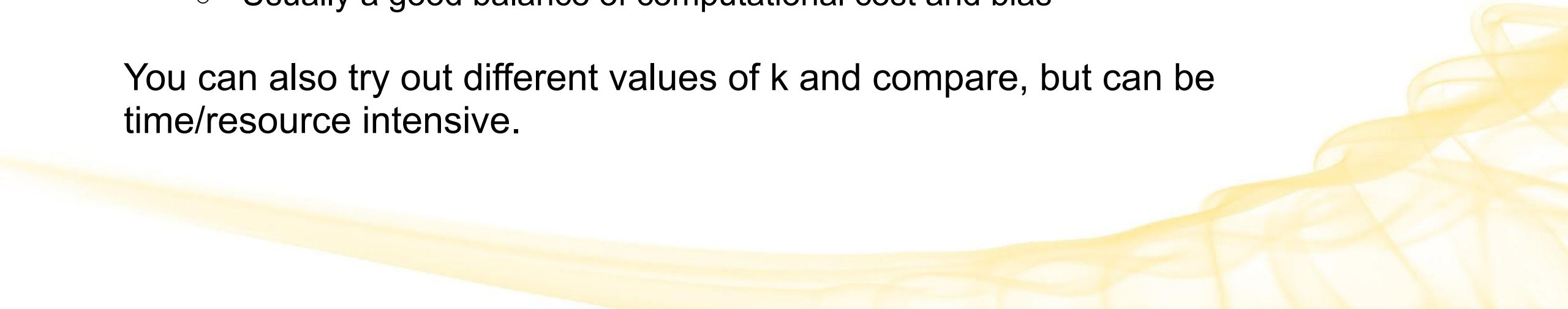
Choice of k is semi-arbitrary.

- Larger k :
 - More computationally expensive, more models needed.
 - Usually less bias, but higher variance
 - Fewer combinations possible

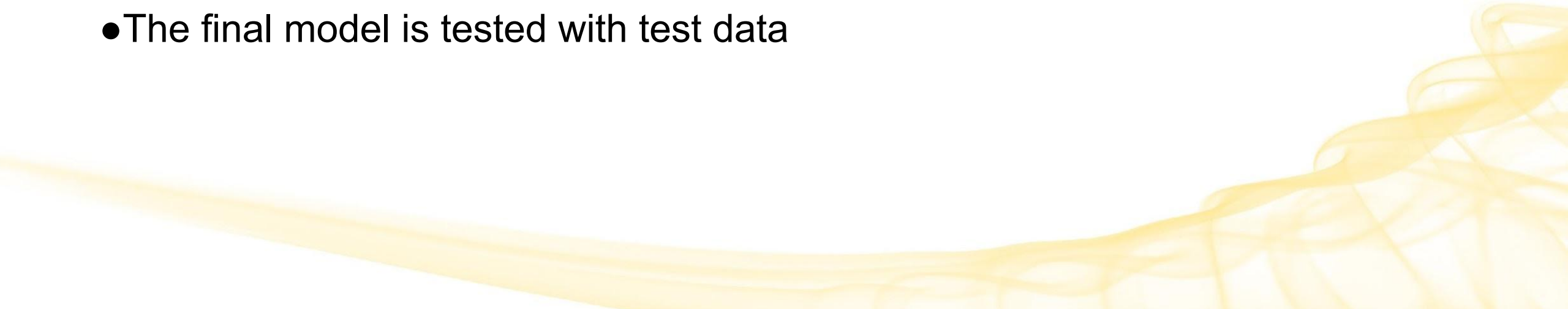
Common choices:

- $k = 5$ folds
- **$k = 10$ folds** → most common
 - Usually a good balance of computational cost and bias

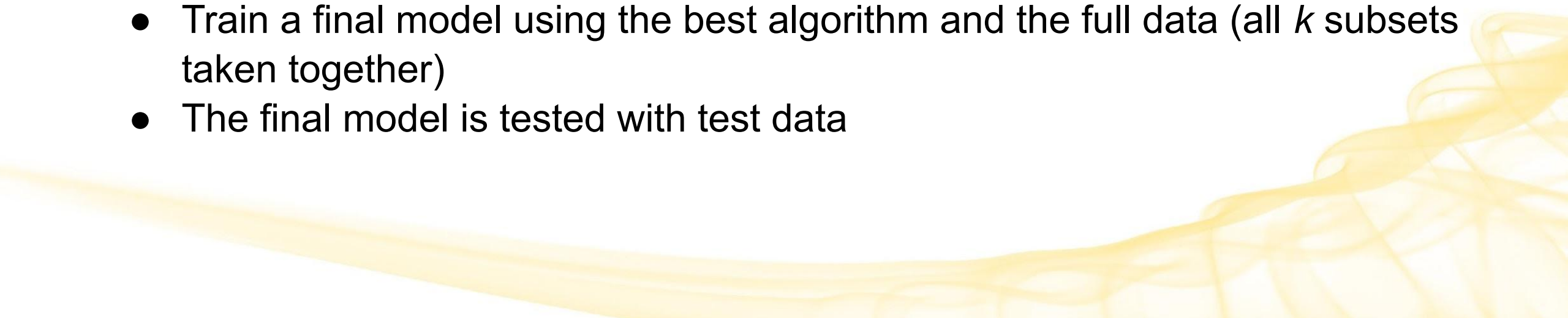
You can also try out different values of k and compare, but can be time/resource intensive.



Algorithm for performance estimation of final model:

- Break the data into k folds
 - Train a model on $k-1$ of these folds, and test on the remaining one.
 - Performance metrics like accuracy are averaged across all iterations to obtain a final estimate. You can also think of these as producing a range of potential accuracy values: $[a_{\min}; a_{\max}]$
 - The full data (all k subsets taken together) is trained on for a final model
 - The final model is tested with test data
- 

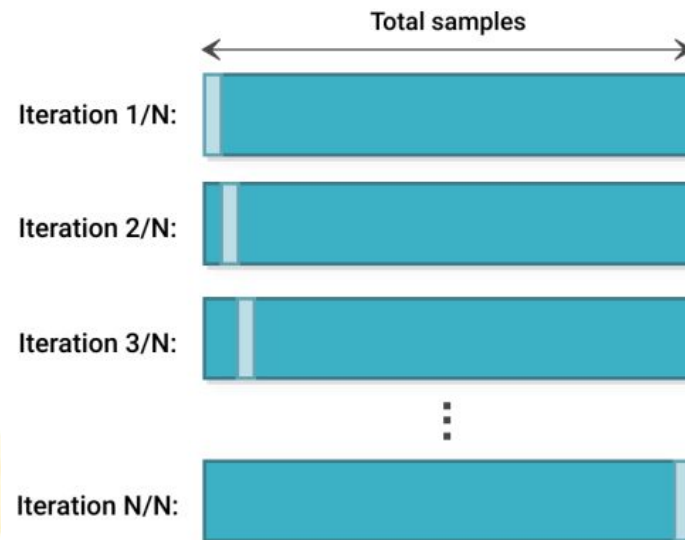
Algorithm for model selection:

- Break the data into k folds
 - For each algorithm (e.g. Decision Trees; 3NN; 5NN):
 - Train a model on $k-1$ of these folds, and test on the remaining one.
 - Performance metrics like accuracy are averaged across all iterations to obtain a final estimate.
 - Choose the best algorithm (e.g. 3NN or 5NN) based on the average accuracy.
 - Train a final model using the best algorithm and the full data (all k subsets taken together)
 - The final model is tested with test data
- 

Leave-One-Out Cross-Validation (LOOCV)

A special case of k-folds CV where $k=n$ (number of examples).

- A model is created and evaluated for **every example** in the training dataset
 - Each example is held out as the validation set while the model is trained on the remaining data ($n-1$ examples).
 - The trained model is evaluated with the 1 example validation “set.”
 - This process is repeated for each data point.
- Performance metrics like accuracy are averaged across all iterations to obtain a final estimate.

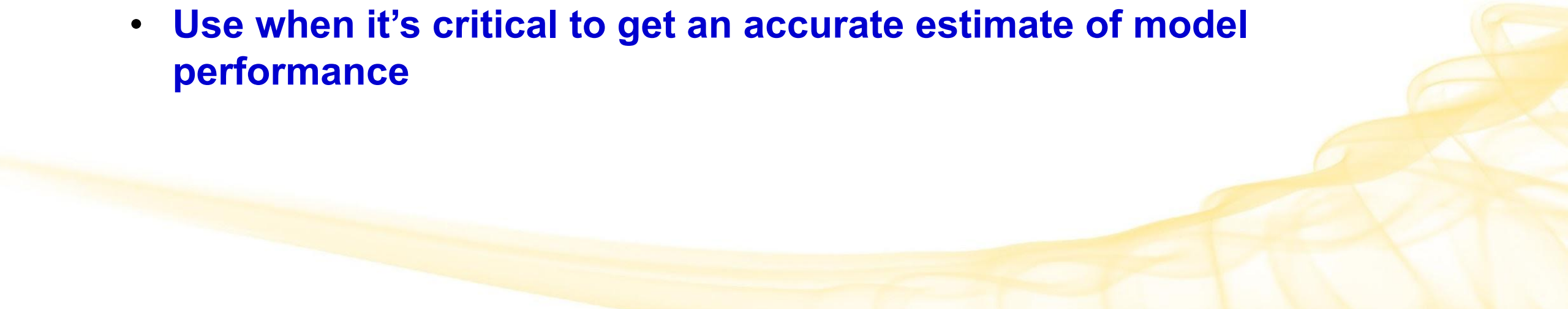


Leave-One-Out Cross-Validation (LOOCV)

LOOCV is computationally very expensive!

- **Don't use on large datasets** (ex. $n > 1000$), or with particularly expensive models

Useful for small datasets (ex. $n < 1000$) because provides a very reliable estimate of model performance.

- Every example is given an opportunity to validate
 - **Use when it's critical to get an accurate estimate of model performance**
- 

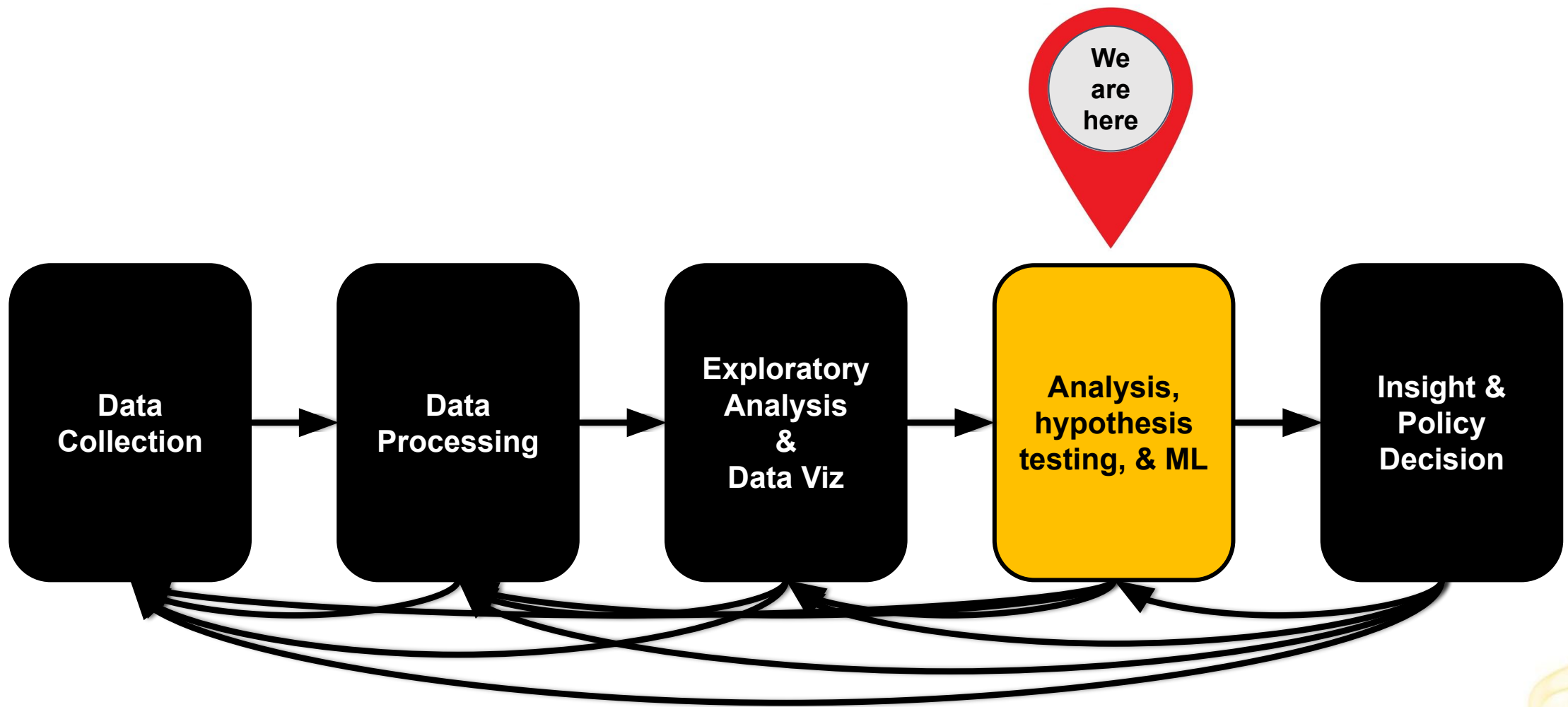
Leave-One-Out Cross-Validation (LOOCV)

Algorithm

1. Split a training dataset into a training set and a validation set.
 - Use 1 example as the validation set.
 - Use the other $n-1$ examples as the training set.
 - You leave one observation “out” from the training set.
 - This is where the method gets the name “leave-one-out” cross-validation.
2. Train a model on the $n-1$ training examples
3. Evaluate the model.
4. Repeat n times, with each data point being used as the test set exactly once.
5. Evaluate across all iterations.

	x_1	x_2	x_3	y	
1					Training Set
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					
16					
17					
18					
19					
20					
21					
22					
23					
24					
25					
26					
27					
28					
29					
30					Testing Set

Decision Trees



Remember: No Free Lunch Theorem (NFL)

There is no single algorithm that will work well for all tasks.



Another Take on Classification

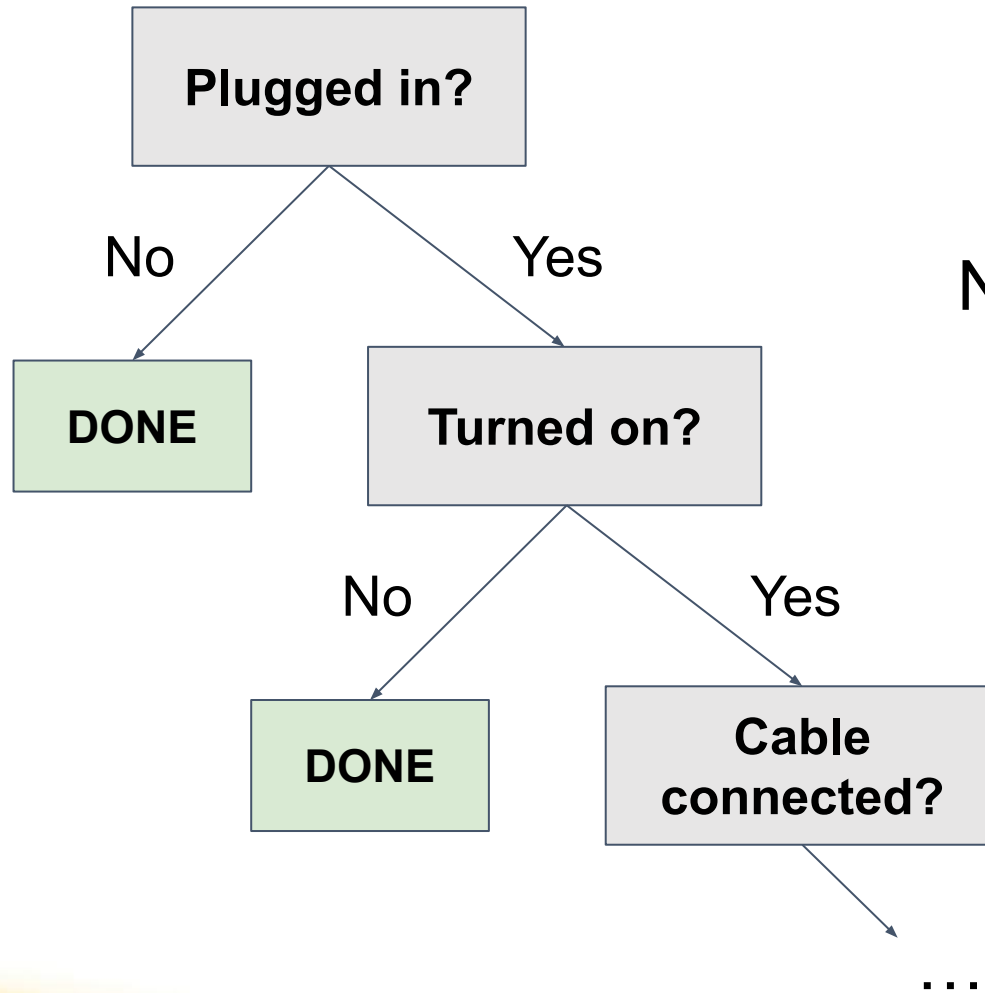
When we looked at the KNN algorithm, we were looking at neighbors as helpful in predicting the label for a new point

Another way to think about the task of classification is to use the training set as a way to create a series of questions. Answering them will get us to the answer for a new point. think of the feature values in an unlabeled example as the answers to a game of questions.

- **Question**
 - Ex. Can this animal fly?
- **A test point value = the answer**
 - Ex. Yes

Given the training data, what is the **best order** in which to ask what questions?

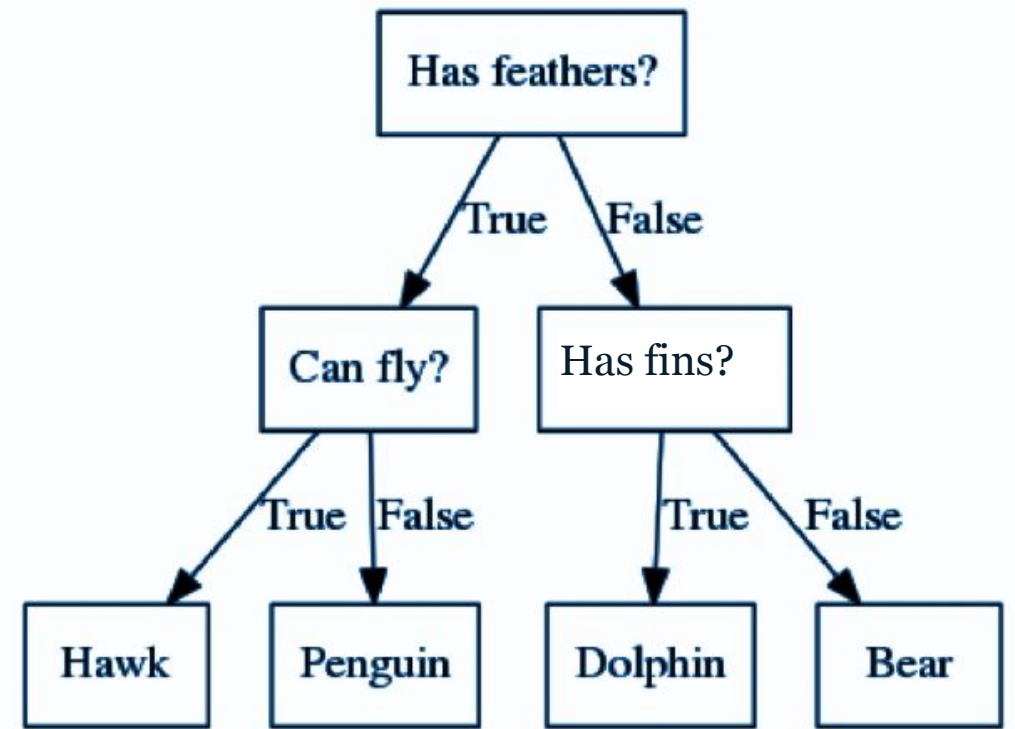
Example: why isn't the TV working?



Notice that this forms a **tree**

Decision Trees (DT)

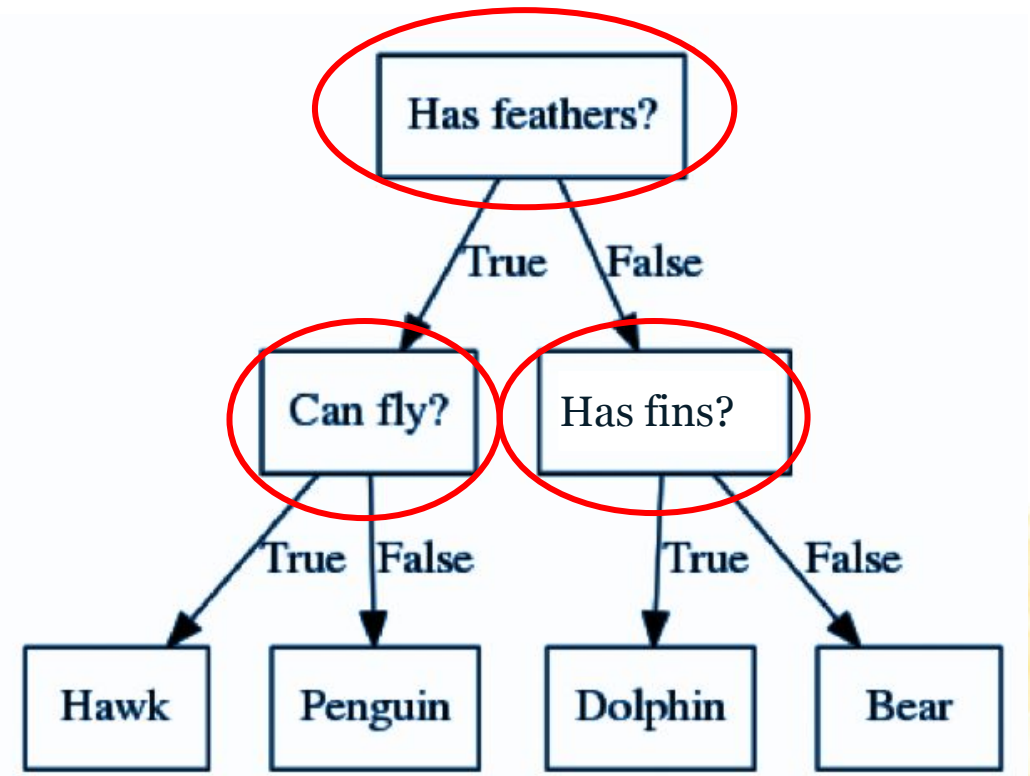
- Define a hierarchy of rules to make a prediction
 - Root and internal nodes test rules
 - Leaf nodes make predictions
- DT learning is about learning such a tree from labeled training data
 - Supervised learning algorithm



Decision Trees (DT)

- Define a hierarchy of rules to make a prediction
 - Root and internal nodes test rules.
 - Leaf nodes make predictions
- DT learning is about learning such a tree from labeled training data
 - Supervised learning algorithm

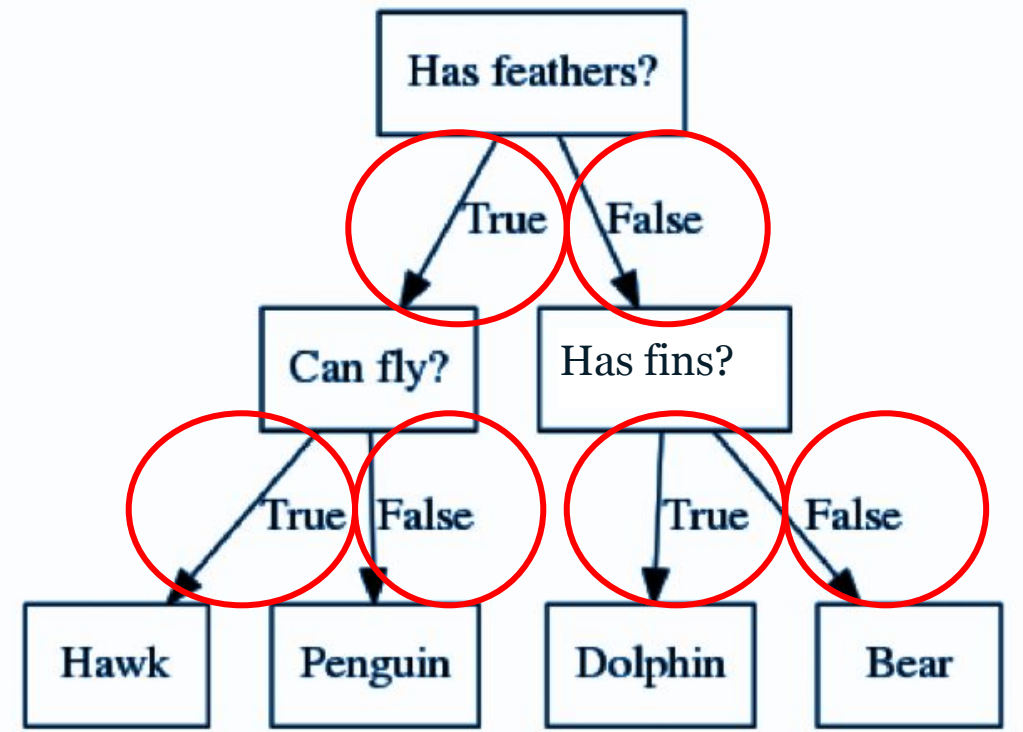
Features are the questions



Decision Trees (DT)

- Define a hierarchy of rules to make a prediction
 - Root and internal nodes test rules.
 - Leaf nodes make predictions
- DT learning is about learning such a tree from labeled training data
 - Supervised learning algorithm

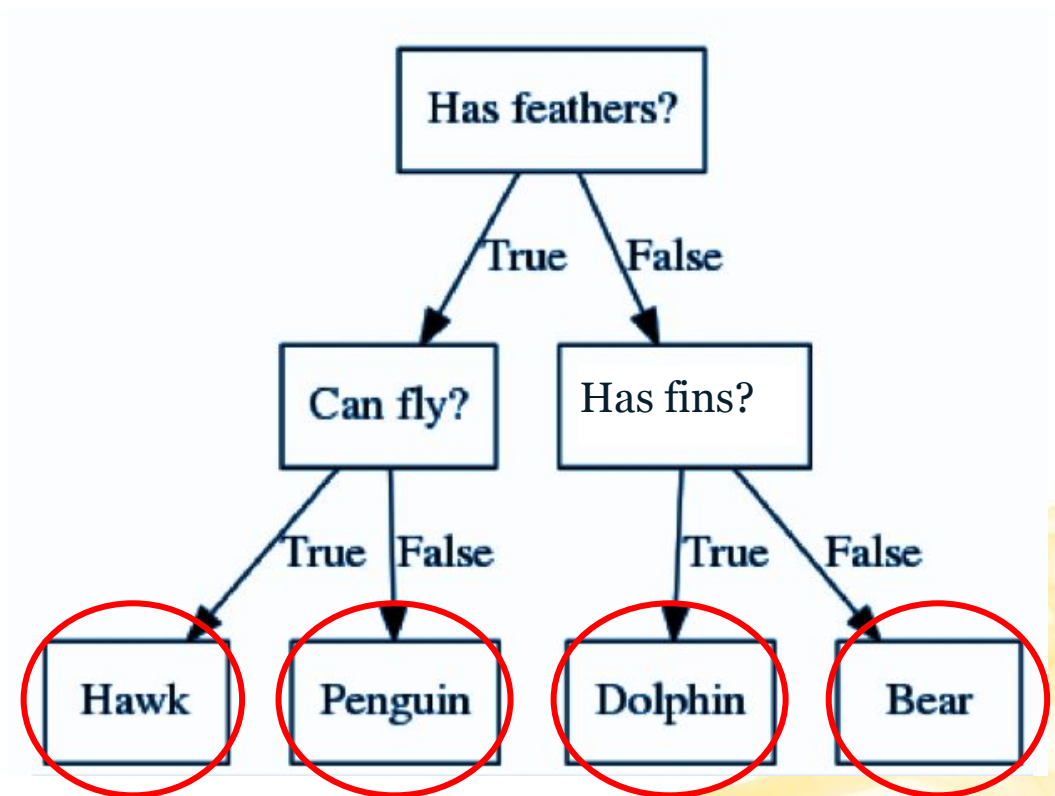
Values are the answers



Decision Trees (DT)

- Define a hierarchy of rules to make a prediction
 - Root and internal nodes test rules.
 - Leaf nodes make predictions
- DT learning is about learning such a tree from labeled training data
 - Supervised learning algorithm

Labels are the predictions



Decision Trees (DT)

- Define a hierarchy of rules to make a prediction
 - Root and internal nodes test rules.
 - Leaf nodes make predictions
- **DT learning is about learning such a tree from labeled training data**
 - Supervised learning algorithm

ID	Has feathers?	Can Fly?	Has Finns?	Class Label
7	True	False	False	<i>Penguin</i>

